

Бази от данни - релационен модел на данните

Ивайло Андреев + chat gpt 5.5, DeepSeek-V3, gemini 3 Flash

25 май 2026

Съдържание

1	Въведение	3
1.1	База данни и СУБД	3
1.2	Модели на бази данни	3
2	Релационен модел на данните	3
2.1	Домейн	3
2.2	Релация	3
2.3	Кортеж	3
2.4	Атрибути	4
2.5	Схема на релация	4
2.6	Схема на релационна база данни	4
2.7	Проектиране на релационна БД	4
2.8	Ключове	5
2.9	Интегритетни ограничения и NULL	5
3	Видове операции	5
3.1	DDL (Data Definition Language)	5
3.2	DML (Data Manipulation Language)	6
4	Заявки към релационна БД	6
4.1	Основни примери	6
5	Тригери	7
5.1	Пример за тригер	7
6	Релационна алгебра	7
6.1	Съвместими релации	7
6.2	Основни операции	8
6.2.1	Обединение (Union)	8

6.2.2	Разлика (Difference)	8
6.2.3	Проекция (Projection)	8
6.2.4	Селекция (Selection)	9
6.2.5	Преименуване (Renaming)	9
6.2.6	Декартово произведение (Cartesian Product)	9
6.3	Допълнителни операции	9
6.3.1	Сечение (Intersection)	9
6.3.2	Съединение (Join)	10
6.3.3	Частно (Division)	10

1 Въведение

1.1 База данни и СУБД

База данни (БД) е организирана колекция от свързани данни, предназначена за ефективно съхранение, търсене и управление в информационни системи.

Система за управление на база данни (СУБД) е софтуерен инструмент, който осигурява средства за дефиниране, съхранение, извличане и манипулиране на данни в БД. Примери: MySQL, PostgreSQL, Oracle, SQL Server, MongoDB.

1.2 Модели на бази данни

Моделът на БД определя логическата структура на данните и начина на тяхната организация. Основните категории са:

- **Релационен модел** — Представя данните като таблици (релации) с редове (кортежи) и колони (атрибути). Манипулацията се извършва чрез SQL. Подходящ за данни със стабилна, строго дефинирана структура.

Нерелационните модели (NoSQL) съществуват като алтернатива, но в тази тема фокусът е върху релационния модел.

2 Релационен модел на данните

2.1 Домейн

Дефиниция 1. Домейн (Област) е множество от допустими стойности за дадена величина. Релационният модел изисква стойностите на всички елементи в кортежите да бъдат **атомарни** (неделими на части).

Домейнът обикновено дефинира скаларния тип данни (INTEGER, VARCHAR, DATE и т.н.) и допълнителни ограничения.

2.2 Релация

Дефиниция 2. Релация е подмножество на декартово произведение $D_1 \times D_2 \times \dots \times D_n$, където D_i са домейни. В контекста на БД релацията представлява таблица с редове (кортежи) и колони (атрибути).

2.3 Кортеж

Дефиниция 3. Кортеж е всеки елемент на релация, т.е. наредена n -торка от стойности. Тъй като релацията е множество, всеки кортеж трябва да е уникален.

Екземпляр на релация е множеството от всички текущо съхранени кортежи. **Кардиналност** е броят на кортежите в екземпляра.

2.4 Атрибути

Дефиниция 4. *Атрибут* е наименование на колона в релация. Всеки атрибут A_i е асоцииран с домейн D_i . Атрибутите описват характеристиките на съхранените обекти.

2.5 Схема на релация

Дефиниция 5. *Схема на релация* R се дефинира чрез име R , множество от атрибути $\{A_1, A_2, \dots, A_n\}$ и съответните домейни. Обозначение: $R(A_1, A_2, \dots, A_n)$.

Свързани понятия:

- **Степен (Arity)** — Броят на атрибутите в схемата. Статична характеристика.
- **Кардиналност** — Броят на кортежите в момента. Динамична характеристика.

2.6 Схема на релационна база данни

Дефиниция 6. *Схема на релационна БД* е съвкупност от всички схеми на релации, включително ограниченията за интегритет (например външни ключове), които управляват връзките между релациите.

Екземпляр на БД е съвкупност от екземплярите на всички релации в дадения момент.

2.7 Проектиране на релационна БД

Проектирането обикновено включва:

1. Определяне какви данни да се съхраняват.
2. Дефиниране на връзки между данните.
3. Определяне на ограничения (ключове, домейни и т.н.).

В практиката се използва **Entity/Relationship (E/R) модел**, който се преобразува към релационна схема на БД чрез:

- Всяка същност (Entity) се превръща в релация със същите атрибути.
- Всяка връзка (Relationship) се превръща в релация, чиито атрибути са ключовете на свързаните същности.

2.8 Ключове

Дефиниция 7. *Суперключ* е множество от атрибути, което еднозначно определя всеки кортеж в релацията.

Дефиниция 8. *Кандидат-ключ* е минимален суперключ.

Дефиниция 9. *Първичен ключ* е избран кандидат-ключ, използван за идентификация на кортежите.

Дефиниция 10. *Външен ключ* е атрибут или множество от атрибути в релация R , които реферират към първичен ключ в друга (или същата) релация.

2.9 Интегритетни ограничения и NULL

- **Entity Integrity** — първичният ключ не може да има NULL стойност.
- **Referential Integrity** — всяка ненулева стойност на външен ключ трябва да съответства на съществуващ ключ в рефрираната релация.

NULL означава липсваща, неизвестна или неприложима стойност. Тя не е число, не е празен низ и не е нула. При сравнения с NULL се използват IS NULL / IS NOT NULL.

3 Видове операции

3.1 DDL (Data Definition Language)

DDL операцияите служат за дефиниране на схемата на БД. Включват CREATE, DROP и ALTER.

```
1  -- Create table Employees
2  CREATE TABLE Employees (
3      employee_id INT PRIMARY KEY,
4      first_name VARCHAR(50) NOT NULL,
5      last_name VARCHAR(50) NOT NULL,
6      salary DECIMAL(10,2),
7      department_id INT
8  );
9
10 -- Add new column
11 ALTER TABLE Employees ADD hire_date DATE;
12
13 -- Drop column
14 ALTER TABLE Employees DROP COLUMN hire_date;
15
16 -- Drop table
17 DROP TABLE Employees;
```

Listing 1: DDL examples

3.2 DML (Data Manipulation Language)

DML операциите служат за манипулация на данните: SELECT, INSERT, UPDATE, DELETE.

```
1 -- Select rows
2 SELECT employee_id, first_name, last_name
3 FROM Employees
4 WHERE salary > 50000;
5
6 -- Insert row
7 INSERT INTO Employees (employee_id, first_name, last_name, salary)
8 VALUES (1, 'John', 'Doe', 75000);
9
10 -- Update rows
11 UPDATE Employees
12 SET salary = 80000
13 WHERE employee_id = 1;
14
15 -- Delete rows
16 DELETE FROM Employees
17 WHERE salary < 30000;
```

Listing 2: DML examples

4 Заявки към релационна БД

4.1 Основни примери

```
1 -- INNER JOIN
2 SELECT
3     e.first_name,
4     e.last_name,
5     d.department_name
6 FROM Employees e
7 INNER JOIN Departments d ON e.department_id = d.department_id
8 WHERE e.salary > 60000;
9
10 -- LEFT JOIN
11 SELECT
12     d.department_name,
13     e.first_name,
14     e.last_name
15 FROM Departments d
16 LEFT JOIN Employees e ON d.department_id = e.department_id
17 ORDER BY d.department_name;
```

Listing 3: SELECT with JOIN

```
1 INSERT INTO Employees (employee_id, first_name, last_name, salary,
2     department_id)
3 VALUES (10, 'Anna', 'Petrova', 3200.00, 3);
```

```

3
4 UPDATE Employees
5 SET salary = 3500.00
6 WHERE employee_id = 10;
7
8 DELETE FROM Employees
9 WHERE employee_id = 10;

```

Listing 4: INSERT, UPDATE, DELETE examples

5 Тригери

Дефиниция 11. *Тригер (Trigger) е специален тип съхранена процедура, която автоматично се активира като отговор на определено събитие (INSERT, UPDATE, DELETE) в дадена таблица.*

5.1 Пример за тригер

```

1 -- Log salary changes into Salary_Log
2 CREATE TRIGGER log_salary_change
3 AFTER UPDATE OF salary ON Employees
4 FOR EACH ROW
5 BEGIN
6     INSERT INTO Salary_Log (employee_id, old_salary, new_salary,
7     change_date)
8     VALUES (NEW.employee_id, OLD.salary, NEW.salary, NOW());
9 END;

```

Listing 5: Trigger: log salary changes

6 Релационна алгебра

Релационна алгебра е математическа система, която дефинира операции над релации. Служи като основа на SQL заявките.

Важно: SQL е практически език, вдъхновен от релационната алгебра, но не е напълно еквивалентен на нея. Аналозиите $\pi \leftrightarrow \text{SELECT}$ и $\sigma \leftrightarrow \text{WHERE}$ са полезни, но не са пълна формална идентичност.

6.1 Съвместими релации

Дефиниция 12. *Релациите $R(A_1, \dots, A_n)$ и $S(B_1, \dots, B_m)$ са съвместими, ако:*

- $n = m$ (еднакъв брой атрибути)
- $\text{dom}(A_i) = \text{dom}(B_i)$ за всеки i (еднакви домейни)
- Редът на атрибутите съвпада

6.2 Основни операции

Основните операции в Релационната алгебра образуват базис и всички останали операции могат да се изразят чрез базисните.

6.2.1 Обединение (Union)

Дефиниция 13. *Обединение $R \cup S$ на съвместими релации R и S съдържа всички кортежи, които се намират в R или S (или в двете).*

Свойства: бинарна, комутативна, асоциативна.

```
1 SELECT employee_id, name FROM full_time_employees
2 UNION
3 SELECT employee_id, name FROM part_time_employees;
```

Listing 6: UNION

6.2.2 Разлика (Difference)

Дефиниция 14. *Разлика $R \setminus S$ съдържа всички кортежи на R , които не се намират в S .*

Свойства: бинарна, некомутиативна.

```
1 SELECT employee_id, name FROM all_employees
2 EXCEPT
3 SELECT employee_id, name FROM terminated_employees;
```

Listing 7: EXCEPT or MINUS

6.2.3 Проекция (Projection)

Дефиниция 15. *Проекция $\pi_{A_{i_1}, \dots, A_{i_k}}(R)$ на релация R с атрибути $\{A_1, \dots, A_n\}$ е релация, съдържаща само избраните атрибути $\{A_{i_1}, \dots, A_{i_k}\}$.*

Еквивалентна е на SQL операцията SELECT с избрани колони.

Ако проекцията не съдържа ключови атрибути, в SQL могат да възникнат дублирани се редове, затова за пълна еквивалентност с релационната алгебра често се използва DISTINCT.

```
1 SELECT DISTINCT first_name, last_name
2 FROM Employees;
```

Listing 8: Projection (SELECT)

6.2.4 Селекция (Selection)

Дефиниция 16. *Селекция $\sigma_P(R)$ е множество от всички кортежи в R , които удовлетворяват предиката P .*

Е еквивалентна на SQL WHERE клаузула.

```
1 SELECT *
2 FROM Employees
3 WHERE salary > 50000 AND department_id = 2;
```

Listing 9: Selection (WHERE)

6.2.5 Преименуване (Renaming)

Дефиниция 17. *Преименуване $\rho_{R'/R}(R)$ или в SQL AS позволяват смяна на имена на релации и атрибути без промяна на данните.*

```
1 SELECT e.employee_id AS emp_id, e.first_name AS name
2 FROM Employees AS e;
```

Listing 10: Renaming

6.2.6 Декартово произведение (Cartesian Product)

Дефиниция 18. *Декартово произведение $R \times S$ съдържа всички възможни комбинации на кортежи от R с кортежи от S .*

Ако R има m редове и S има n редове, $R \times S$ има $m \times n$ редове.

В релационната алгебра атрибути с едно и също име в различни релации трябва формално да се преименуват преди декартовото произведение между тях. В SQL това е позволено, но при проекция трябва да се уточни към коя таблица принадлежи колоната.

```
1 SELECT e.employee_id, d.department_id
2 FROM Employees e, Departments d;
3 -- Or: FROM Employees e CROSS JOIN Departments d;
```

Listing 11: Cartesian product

6.3 Допълнителни операции

6.3.1 Сечение (Intersection)

Дефиниция 19. *Сечение $R \cap S$ съдържа само кортежите, които се намират едновременно в R и S .*

Свойства: комутативна, асоциативна. Може да се изрази чрез разлика: $R \cap S = R \setminus (R \setminus S)$.

```

1 SELECT employee_id FROM employees_2023
2 INTERSECT
3 SELECT employee_id FROM employees_2024;

```

Listing 12: INTERSECT

6.3.2 Съединение (Join)

Дефиниция 20. *Theta Join* $R \bowtie_P S$ е множество от всички кортежи от $R \times S$, които удовлетворяват предиката P .

Може да се изрази чрез декартово произведение и селекция: $R \bowtie_P S = \sigma_P(R \times S)$.

Естествено съединение $R \bowtie S$ е частен случай на theta join, където съединяваме релации по атрибути с еднакви имена. Резултатът съдържа само един екземпляр от общите атрибути.

```

1 -- Theta Join (INNER JOIN)
2 SELECT *
3 FROM Employees e, Departments d
4 WHERE e.department_id = d.department_id;
5
6 -- Equivalent form:
7 SELECT *
8 FROM Employees e
9 INNER JOIN Departments d ON e.department_id = d.department_id;
10
11 -- Natural join (same attribute names)
12 SELECT *
13 FROM Employees
14 JOIN Departments USING (department_id);

```

Listing 13: JOIN operations

6.3.3 Частно (Division)

Дефиниция 21. *Частно* $R \div S$ (където атрибутите на S са подмножество от атрибутите на R) е релация, съдържаща всички кортежи от $\pi_{\text{останалите атрибути}}(R)$, за които всяка комбинация с кортеж от S съществува в R .

Частното е свързано с декартовото произведение в смисъл, че ако $R \div S = T$, то винаги е в сила включването $T \times S \subseteq R$. Равенството $T \times S = R$ е изпълнено само при специален случай, когато R съдържа всички комбинации между кортежите на T и S .

Интуиция: Ако R съдържа $(student_id, sport)$, а S съдържа $(sport)$, то $R \div S$ дава студентите, които практикуват всички спортове в S .

Пример:

StudentSports			
student_id	sport	Sports	
1	badminton	sport	
2	cricket	badminton	
2	badminton	cricket	
4	cricket		

StudentSports ÷ Sports	
student_id	
2	

Резултатът съдържа само студент 2, защото той е единственият студент, който практикува всички спортове от релацията Sports.